

# SIMATS ENGINEERING

## ASSESSMENT TOOL 2: Error Analysis Task

**COURSE NAME:** Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

---

### COURSE OUTCOME COVERED

**CO5:** Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN. (BL5)

Dave's Taxonomy: Articulation (Level 4)  
Question Count: 5

**Total Marks: 25**

---

### Code of Conduct

I \_\_\_\_\_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: \_\_\_\_\_

### Assessment Tasks

#### Error Analysis Task

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.
2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.
3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.
4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.
5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

### Error Analysis Task Rubric

| Criteria                | Weightage | Level 4 - Exemplary                                 | Level 3 - Proficient         | Level 2 - Developing       | Level 1 - Beginning     |
|-------------------------|-----------|-----------------------------------------------------|------------------------------|----------------------------|-------------------------|
| Identification of Error | 30%       | Accurately identifies the root technical issue      | Identifies most of the issue | Partially identifies issue | Fails to identify issue |
| Cause Analysis          | 20%       | Explains root cause with strong technical reasoning | Reasonable cause analysis    | Basic cause analysis       | Weak analysis           |

|                                      |     |                                             |                                |                     |                         |
|--------------------------------------|-----|---------------------------------------------|--------------------------------|---------------------|-------------------------|
| Corrective Action                    | 25% | Proposes effective and feasible corrections | Reasonable corrections         | Basic corrections   | Ineffective corrections |
| Use of Hadoop / Integration Concepts | 15% | Applies relevant concepts appropriately     | Mostly appropriate application | Partial application | Incorrect application   |
| Clarity                              | 10% | Clear and direct explanation                | Generally clear                | Somewhat unclear    | Poorly presented        |

## SOLUTION

### Task 1: Uneven Input Splits and Idle Reducers

#### Error Identification

- **Root Issue:** Severe data skew during the Map-side splitting and default HashPartitioner routing on non-uniformly distributed keys.

#### Cause Analysis

- **Map Phase:** Non-splittable compression formats (e.g., standard gzip) or many small files force a few mappers to process disproportionately massive workloads while others finish instantly.
- **Reduce Phase:** Default HashPartitioner maps high-frequency keys (e.g., null or common status codes) to a single reducer, causing that worker to choke while remaining reducers sit idle.

#### Corrective Actions

- Implement splittable compression codecs like **Bzip2** or **Snappy with container formats** (SequenceFile/Parquet/ORC).
- Use CombineFileInputFormat to merge small files into uniform splits before mapper invocation.
- Write a **Custom Partitioner** or use key salting (prefixing random integers to hot keys) to uniformly distribute data across all reducers.
- Enable speculative execution (mapreduce.reduce.speculative = true) to detect and duplicate lagging reducer tasks.

### Task 2: HDFS Data Unavailability After Single Node Failure

#### Error Identification

- **Root Issue:** Replication factor misconfiguration ( $R = 1$ ) and/or lack of NameNode High Availability (Single Point of Failure).

#### Cause Analysis

- **Data Layer:** The default replication factor was overridden (dfs.replication = 1) or blocks were placed without rack awareness. When the single DataNode holding those unique blocks goes down, those files become immediately corrupted/unavailable.
- **Metadata Layer:** A single active NameNode deployment without a standby instance causes the entire filesystem metadata to become unreachable if that master node experiences hardware or network failure.

#### Corrective Actions

- Reset dfs.replication to the standard value of **3** and execute `hdfs fsck / -reproduce` to identify and re-replicate under-replicated blocks.
- Deploy **HDFS NameNode High Availability (HA)** using active/standby instances backed by ZooKeeper Failover Controllers (ZKFC) and a Quorum Journal Manager (QJM) cluster.
- Configure dfs.network.topology.impl to enforce **Rack Awareness**, ensuring replicas are split across at least two distinct failure domains.

### Task 3: Duplicate Record Creation in NiFi Ingestion Pipeline

#### Error Identification

- **Root Issue:** At-least-once delivery semantics coupled with non-idempotent consumer writes and lack of transactional state tracking.

#### Cause Analysis

- **Pipeline Retries:** Upstream processor failures, network timeouts, or uncommitted flow files in NiFi trigger automatic retries, resending previously ingested batches.
- **Non-Atomic Sinks:** Downstream targets (e.g., Hive, HBase, RDBMS) perform unconstrained INSERT statements rather than upserts or deduplication checks.

#### **Corrective Actions**

- Implement **Distributed State Management** in NiFi (DistributedMapCacheServer / DistributedMapCacheClient) with a DetectDuplicate processor based on message payload hashes (SHA-256).
- Switch downstream storage engines to ACID-compliant transactional tables (e.g., Apache Iceberg, Delta Lake, or Hive ACID) utilizing MERGE INTO / upsert statements.
- Configure idempotent write mechanisms using unique business keys as explicit primary or row keys.

### **Task 4: YARN Resource Starvation Under Multi-User Workloads**

#### **Error Identification**

- **Root Issue:** Misconfigured YARN FIFO Scheduler (First-In, First-Out) or Capacity Scheduler without memory/core limits and preemption.

#### **Cause Analysis**

- Under default FIFO scheduling, the first large job monopolizes all available cluster memory and vCores, placing all subsequent user requests into a blocked waiting queue.
- Under default Fair/Capacity setups without user limits, single-user unbounded jobs consume the entire queue capacity without resource caps.

#### **Corrective Actions**

- Switch from FIFO to the **Capacity Scheduler** or **Fair Scheduler** (yarn.resourcemanager.scheduler.class).
- Configure hierarchical queues with hard boundaries:
  - Define minimum and maximum allocation percentages per tenant queue (yarn.scheduler.capacity.root.<queue-path>.capacity and maximum-capacity).
  - Set yarn.scheduler.capacity.root.<queue-path>.user-limit-factor = 1 to prevent single-user cluster hijacking.
- Enable scheduler preemption (yarn.resourcemanager.scheduler.monitor.enable = true) to reclaim containers from low-priority tasks when high-priority jobs arrive.

### **Task 5: Backup Strategy Restoring Outdated Data**

#### **Error Identification**

- **Root Issue:** Asynchronous replication lag, non-atomic backup mechanisms, or restoring from stale file-copy snapshots rather than consistent metadata points.

#### **Cause Analysis**

- **Metadata Inconsistency:** Running file-level backup tools (like DistCp) directly on active, constantly mutating HDFS directories captures an inconsistent state where file headers, data blocks, and edit logs do not align.
- **Lagging Asynchronous Mirrors:** Standby clusters replicated via loose schedule intervals (e.g., daily batches) miss real-time transactional logs and edits occurring between sync cycles.

#### **Corrective Actions**

- Enforce read-only **HDFS Point-in-Time Snapshots** (hdfs dfsadmin -allowSnapshot and hdfs dfs -createSnapshot) before triggering backup operations to guarantee data freeze and atomicity.
- Use DistCp to copy exclusively from the generated snapshot path rather than live directories (hdfs dfs -distcp hdfs://cluster/.snapshot/s1 hdfs://backup-cluster/).
- Implement transactional snapshotting tools like **Apache Falcon** or table-native time-travel features (Iceberg/Delta log commits) to guarantee point-in-time recovery to the exact recovery point objective (RPO).